

Machine Learning Assignment 1: Polynomial Regression

Pranay Sanjan Kanneti

1 Introduction and Approach

This report documents the approach used to develop polynomial regression models for two geothermal energy datasets. The primary goal was to predict a continuous target variable y while avoiding underfitting and overfitting.

The core model, including the closed-form matrix operations for L2 regularization (Ridge Regression) and the evaluation metrics (MSE and R^2), was built using standard `numpy`. I used `scikit-learn` for KFold algorithm, data preparation tasks, specifically for splitting the data and generating polynomial combinations (`PolynomialFeatures`).

While writing the code, my priority was on correctness and thorough validation over runtime. Even though the validation R^2 started dropping after the optimal complexity, I intentionally let the grid search run through all degrees up to 10 to ensure a complete evaluation (in context of model1).

2 Methodology

To find the best polynomial degree, I needed a reliable way to test the models on unseen data.

Initially, I tested a simple 80-20 train-validation split. This straightforward method worked fairly well and achieved a validation R^2 score of 0.92 on the first try for the first model.

However, to make the results more reliable and unbiased, I moved to 5-Fold Cross-Validation. In 5-Fold Cross-Validation, the model trains and tests five different times on different slices of the data, and the errors are averaged.

I used L2 Regularization (Ridge Regression) primarily because it allows a direct, closed-form matrix solution $\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$. Adding the $\lambda \mathbf{I}$ term ensures that the matrix is always invertible and avoids numerical singularity, making training a straightforward matrix solve in NumPy without needing iterative optimization. Furthermore, as higher polynomial degrees create correlated terms, the L2 penalty naturally shrinks these weights together to keep the fitted curve smooth and prevent overfitting.

(NOTE : Instead of manually computing an explicit matrix inverse, I solved the regularized normal equations directly using `np.linalg.solve`).

```

Running Train-validation split Grid Search...
Degrees: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Lambdas: [0.0001, 0.001, 0.01, 0.1, 0, 1.0, 10.0]
-----
Optimal Degree:      5
Optimal Lambda:      0.01
Training MSE:        1.4391e-01
Validation MSE:       6.8915e-01
Validation R2:        0.9232
Total Features:      462
=====

Top 10 configurations ranked by validation MSE:
degree  lambda  train_mse  val_mse  val_r2  total_features
-----
5  0.0100  0.143905  0.689146  0.923220  462
5  0.1000  0.237708  0.699350  0.922083  462
6  0.1000  0.172957  0.831037  0.907411  924
4  0.0100  0.396797  0.889124  0.900940  210
4  0.0010  0.383306  0.913432  0.898231  210
4  0.0001  0.382963  0.920503  0.897443  210
4  0.0000  0.382959  0.921435  0.897340  210
4  0.1000  0.522862  0.930854  0.896290  210
7  1.0000  0.315647  0.978741  0.890955  1716
6  1.0000  0.438076  0.983876  0.890383  924
=====

```

Figure 1: Initial test using a simple 80-20 train-validation split on model1.

3 Phase 1: Power Plant Steam Turbine Optimization (var1)

For the first phase, the dataset contained 6 operational parameters to predict the Net Power Score.

Upon inspecting the dataset, I observed that all the raw feature values for these parameters naturally fell strictly between -1 and 1. Because the data was already perfectly bounded, I decided not to apply any feature scaling.

To find the best model, I tested degrees up to 10 using a simple tuning process:

- First, I tested a broad set of λ values from 10^{-4} to 10.0 across all degrees. This showed that **Degree 5** worked best, with the optimal λ lying close to 0.01 (Validation MSE ≈ 0.5389).
- Next, I zoomed in and tested smaller intervals between 0.002 and 0.009 around this region to fine-tune the penalty, and found the optimal λ to be around 0.004.

```

Running 5-Fold Cross-Validation Grid Search...
Degrees: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Lambdas: [0.0001, 0.001, 0.01, 0.1, 0, 1.0, 10.0]
-----
Top 5 Configurations (Ranked by Validation MSE):
degree  lambda  train_mse  val_mse  val_r2  total_features
-----
5  0.010  0.265393  0.538940  0.943975  462
5  0.001  0.151665  0.556515  0.942048  462
6  0.010  0.223481  0.570226  0.940715  924
7  0.010  0.170369  0.573909  0.940240  1716
8  0.010  0.156008  0.642997  0.933007  3003
=====

Optimal Degree:      5
Optimal Lambda:      0.01
Best Validation MSE:  5.3894e-01
Corresponding Val R2: 0.9440
=====

```

Figure 2: Initial 5-Fold CV coarse grid search across wide λ values for Phase 1.

Through this progressive refinement, the grid search converged on an optimal configuration of **Degree 5** with a regularization parameter of $\lambda = 0.0036$. As shown in the final terminal results, this tuned model further reduced the Best Validation MSE to 0.5055

```

Running 5-Fold Cross-Validation Grid Search...
Degrees: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Lambdas: [0.0033, 0.0034, 0.0035, 0.0036, 0.0037]
-----

Top 5 Configurations (Ranked by Validation MSE):
degree  lambda  train_mse  val_mse  val_r2  total_features
5      0.0036   0.195357  0.505464  0.947408      462
5      0.0035   0.193975  0.505464  0.947407      462
5      0.0037   0.196725  0.505510  0.947405      462
5      0.0034   0.192578  0.505515  0.947401      462
5      0.0033   0.191165  0.505621  0.947389      462

=====
Optimal Degree:      5
Optimal Lambda:      0.0036
Best Validation MSE: 5.0546e-01
Corresponding Val R2: 0.9474
=====

```

Figure 3: Final refined 5-Fold CV terminal output showing the optimal tuned parameters for Phase 1.

(5.0546×10^{-1}) and improved the validation R^2 score to 0.9474 across 462 polynomial features.

4 Phase 2: Subterranean Thermal Reservoir Mapping (var2)

For the second phase, the goal was to predict a Thermal Anomaly Score using 3 spatial coordinate offsets.

Similar to Phase 1, the feature values for this dataset were also strictly bounded between -1 and 1. Therefore, I again skipped feature scaling to preserve the natural boundaries of the spatial coordinates. I searched for the optimal model up to degree 20.

```

Running 5-Fold Cross-Validation Grid Search...
Degrees: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
Lambdas: [0.0001, 0.001, 0.01, 0.1, 0.005, 0, 1.0, 10.0]
-----

Top 5 Configurations (Ranked by Validation MSE):
degree  lambda  train_mse  val_mse  val_r2  total_features
14      0.0010   0.177089  0.254841  0.994052      680
16      0.0010   0.171456  0.256830  0.994006      969
10      0.0001   0.163068  0.257734  0.993951      286
15      0.0010   0.174482  0.259927  0.993935      816
12      0.0001   0.151337  0.260295  0.993893      455

=====
Optimal Degree:      14
Optimal Lambda:      0.001
Best Validation MSE: 2.5484e-01
Corresponding Val R2: 0.9941
=====

```

Figure 4: Terminal output showing 5-Fold CV results for Phase 2.

Using 5-fold cross-validation, degree 14 with $\lambda = 0.001$ achieved the lowest mean validation MSE of 0.254841 and the highest corresponding validation R^2 of 0.994052. Although degree 10 gave comparable performance, degree 14 was chosen based on the validation criterion. Since L2 regularization was actively shrinking unnecessary weights, the higher degree did not lead to overfitting and still generalized effectively.

5 GitHub Repository Structure

The complete code, training notebooks, and submission files are available at: <https://github.com/sanjus09/ML-Assignment1>.

The repository is structured as follows:

- `data/`: Contains the provided training CSV files.
- `model1.ipynb` & `model2.ipynb`: The main Jupyter Notebooks containing the complete, end-to-end pipelines for Phase 1 and Phase 2. Both notebooks share the same core mathematical logic but differ in their input/output files and hyperparameter search ranges.
- `prediction_files/`: Contains the final exported prediction CSV files formatted for submission.
- `Test_files/`: Contains early experimental scripts and rough tests. *The evaluator can safely ignore this folder.*
- `Report.pdf`

6 Conclusion

Overall, through 5-fold cross-validation along with L2 regularization helped lock down the right degree and penalty for both problems without messing with feature scaling. Using the best hyperparameters found (degree 5 for Phase 1 and degree 14 for Phase 2), I retrained the final weights on the full training data and saved the test predictions to the submission CSVs.